HOSPITAL EMERGENCY RESPONSE CO-ORDINATOR

SOURCE CODE

Priority Process

```
#include "hospital_messages.h"


static void calculate_priority(int severity, priority_info_t *prio) {
    if (severity >= 8) {
        prio->level = PRIORITY_P1_CRITICAL;
        strncpy(prio->level_str, "P1 - CRITICAL", sizeof(prio->level_str) - 1);
        strncpy(prio->action, "SEND TO ICU", sizeof(prio->action) - 1);
        strncpy(prio->department, "ICU", sizeof(prio->department) - 1);
    } else if (severity >= 6) {
        prio->level = PRIORITY_P2_HIGH;
        strncpy(prio->level_str, "P2 - HIGH", sizeof(prio->level_str) - 1);
        strncpy(prio->action, "SEND TO GENERAL BED", sizeof(prio->action) - 1);
        strncpy(prio->department, "GENERAL BED", sizeof(prio->department) - 1);
    } else if (severity >= 3) {
        prio->level = PRIORITY_P3_MEDIUM;
        strncpy(prio->level_str, "P3 - MEDIUM", sizeof(prio->level_str) - 1);
        strncpy(prio->action, "SEND TO GENERAL BED", sizeof(prio->action) - 1);
        strncpy(prio->department, "GENERAL BED", sizeof(prio->department) - 1);
    } else {
        prio->level = PRIORITY_P4_LOW;
        strncpy(prio->level_str, "P4 - LOW", sizeof(prio->level_str) - 1);
        strncpy(prio->action, "SEND TO OUTPATIENT", sizeof(prio->action) - 1);
        strncpy(prio->department, "OUTPATIENT", sizeof(prio->department) - 1);
    }
```

```
}

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    name_attach_t *attach = name_attach(NULL, ATTACH_POINT_PRIORITY, 0);
    if (attach == NULL) {
        perror("[PriorityProcess] name_attach failed");
        return EXIT_FAILURE;
    }

    printf("[PriorityProcess] Online and listening on '%s' (chid=%d)\n",
           ATTACH_POINT_PRIORITY, attach->chid);
    fflush(stdout);

    int resource_coid = -1;
    hospital_msg_t msg;
    hospital_reply_t rep;
    int rcvid;

    while (1) {
        rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);
        if (rcvid == -1) {
            perror("[PriorityProcess] MsgReceive error");
            continue;
        }
        if (rcvid == 0) {
            continue;
        }
```

```
    if (msg.msg_type == MSG_TYPE_BATCH_DONE) {
        printf("[PriorityProcess] Batch completion signal received. Forwarding to
ResourceProcess...\n");
        fflush(stdout);

        if (resource_coid == -1) {
            resource_coid = name_open(ATTACH_POINT_RESOURCE, 0);
        }
        if (resource_coid != -1) {
            hospital_reply_t fwd_rep;
            MsgSend(resource_coid, &msg, sizeof(msg), &fwd_rep, sizeof(fwd_rep));
        }

        memset(&rep, 0, sizeof(rep));
        rep.msg_type = MSG_TYPE_REPLY_OK;
        rep.status = 0;
        strncpy(rep.message, "Batch forwarded", sizeof(rep.message) - 1);
        MsgReply(rcvid, EOK, &rep, sizeof(rep));
        continue;
    }
    calculate_priority(msg.record.patient.severity, &msg.record.priority);
    printf("[PriorityProcess] Patient %s (%s, Sev:%d) -> %s -> %s\n",
           msg.record.patient.patient_id,
           msg.record.patient.condition,
           msg.record.patient.severity,
           msg.record.priority.level_str,
           msg.record.priority.action);
```

```
        fflush(stdout);
        if (resource_coid == -1) {
            resource_coid = name_open(ATTACH_POINT_RESOURCE, 0);
            if (resource_coid == -1) {
                printf("[PriorityProcess] Warning: Could not connect to '%s' yet (errno=%d). Retrying...\n",
                        ATTACH_POINT_RESOURCE, errno);
                fflush(stdout);
            }
        }
        if (resource_coid != -1) {
            msg.msg_type = MSG_TYPE_RESOURCE_ALLOC;
            hospital_reply_t res_rep;
            if (MsgSend(resource_coid, &msg, sizeof(msg), &res_rep, sizeof(res_rep)) == -1) {
                perror("[PriorityProcess] MsgSend to ResourceProcess failed");
                name_close(resource_coid);
                resource_coid = -1;
            }
        }
        memset(&rep, 0, sizeof(rep));
        rep.msg_type = MSG_TYPE_REPLY_OK;
        rep.status = 0;
        strncpy(rep.message, "Priority Calculated & Forwarded", sizeof(rep.message) - 1);
        MsgReply(rcvid, EOK, &rep, sizeof(rep));
    }
    name_detach(attach, 0);
    return EXIT_SUCCESS;
}
```